Lecture 9-11 AVL Trees

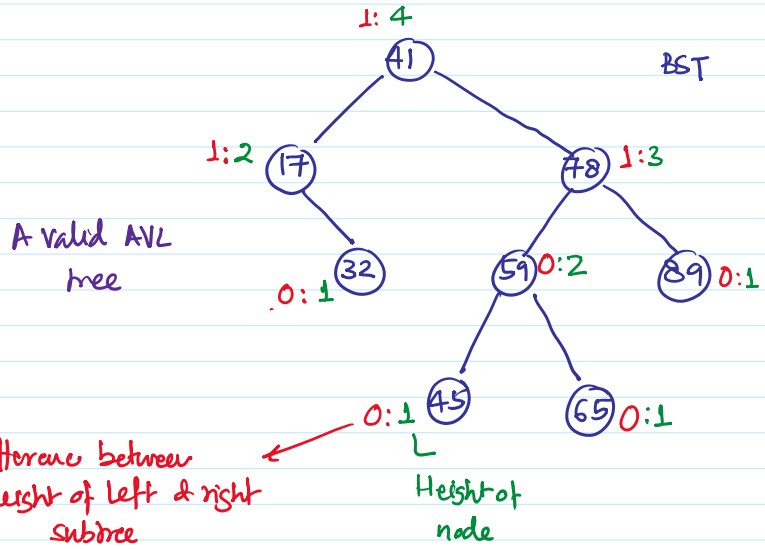
14 August 2026 11:50

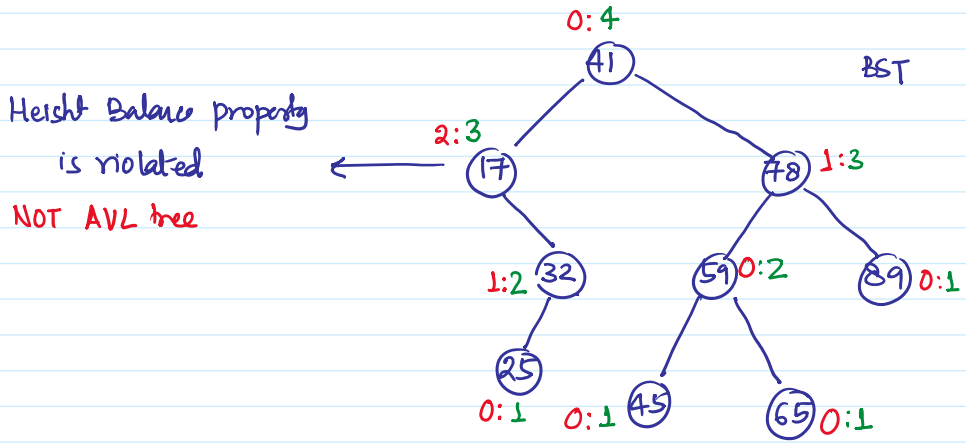
Binary Search Tree over n nodes — Search, Insertion, Deletion — $O(h)$ — h can be $O(n)$ in worst-case

AVL tree - (i) should be BST T

(ii) $\forall$ internal nodes v of T the difference between height of left subtree of v and right subtree of v should be at most 1.

All leaves are at height 1





Height of an AVL tree

Prop: Height of AVL tree T over n nodes is $O(\log n)$.

Proof:

$n(h) :=$ min no. of nodes in an AVL tree of height h.

We will compute $n(h)$

$n(1) = 1$ $\quad\quad$ $n(2) = 2$



For $h \geq 3$, an AVL tree of height h contains one root node

- one AVL tree of height $h-1$
- one AVL tree of height $(h-1)$ or $(h-2)$

$$n(h) = 1 + n(h-1) + n(h-2)$$

$$\geq n(h-1) + n(h-2) \quad \because \; n(h-1) \geq n(h-2)$$

$$n(h) \geq 2\, n(h-2)$$

$$\geq 2 \cdot 2\, n(h-4)$$

$$\geq 2 \cdot 2 \cdot 2\, n(h-6)$$

$$\vdots$$

$$n(h) \geq 2^i\, n(h-2i) \qquad i = \frac{h}{2} - 1 \quad \Rightarrow \quad h - 2i = 2$$

$$\geq 2^{\left(\frac{h}{2}-1\right)} n(2)$$

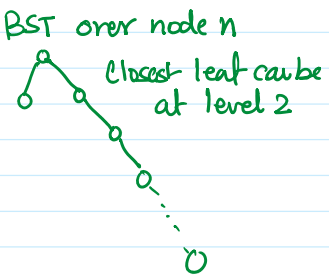
$$\Rightarrow \quad n(h) \geq 2^{\left(\frac{h}{2}-1\right)} \cdot 2 = 2^{\frac{h}{2}}$$

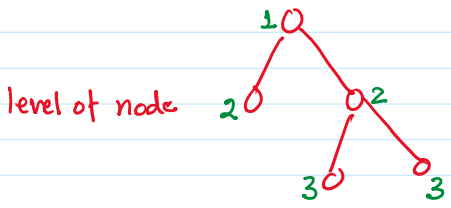
$$\Rightarrow \boxed{h \leq 2 \log n(h)}$$

Height of AVL tree is $O(\log n)$

Properties of AVL tree

Prop Consider an AVL tree over n nodes. Consider a leaf which is closest to root. Suppose the leaf is at level k. Then height of AVL tree is atmost $2k-1$.

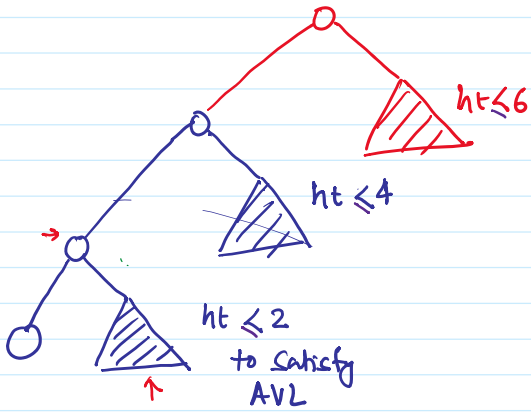




Proof

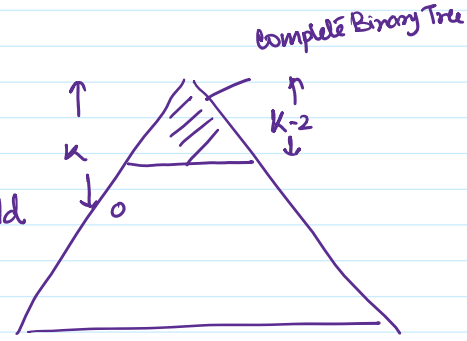
K	Upper Bound on height
2	3
3	5
4	7

$ht(T) \leq 2K-1$



Proposition If the closest leaf is at level K, then all the nodes from level 1 .. (K-2) have two children.

Proof By Contradiction



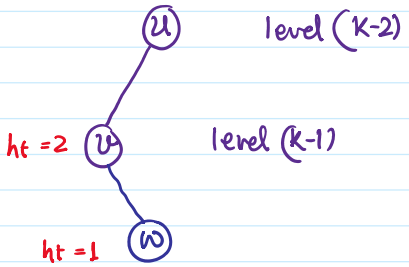
Suppose ∃ a node u at level (K-2) has only one child say v

Since v is level (K-1) so it can't be a leaf node (By assumption)

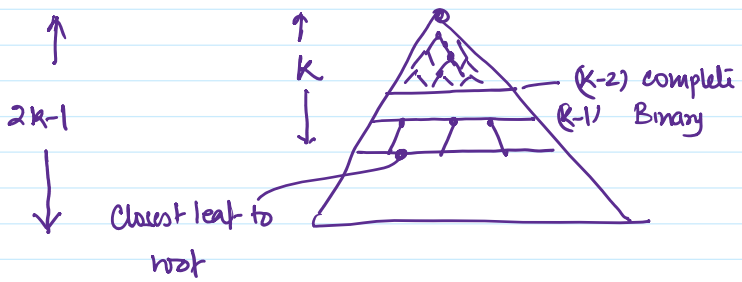
let w be a child of v

⇒ Height Balance property is violated at u

Contradiction



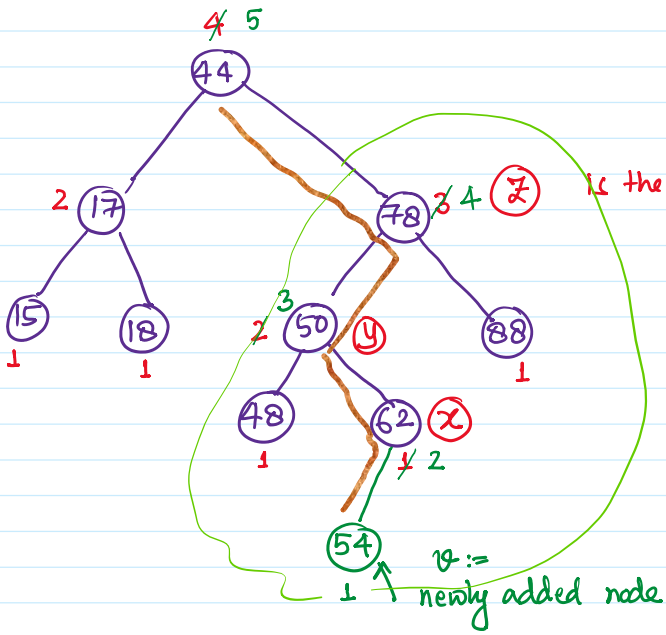
If the closest leaf is at level K, then node at level (K-1) can also have one child. But that child should be a leaf node. Or height Balance property will be violated.



Insertion in AVL tree

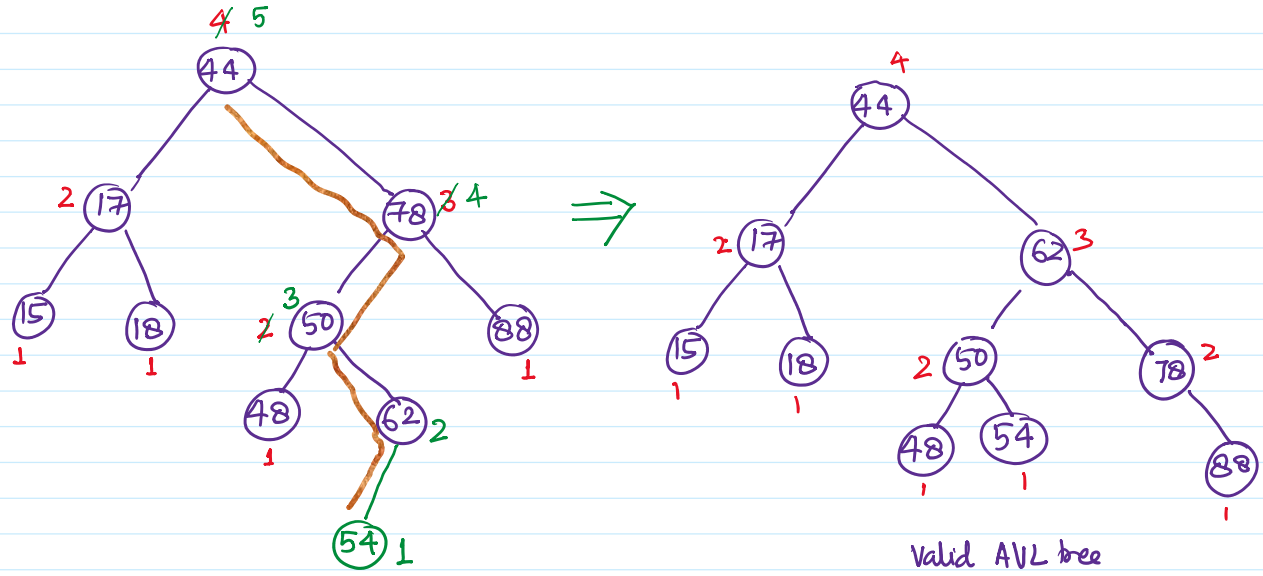
Insertion in AVL tree

- Inserting a node in BST is easy. Need to find appropriate position and add node there.
- Inserting a node v in AVL tree can change height of some node in tree.



is the first node on path from v to root on which height balance property is violated.

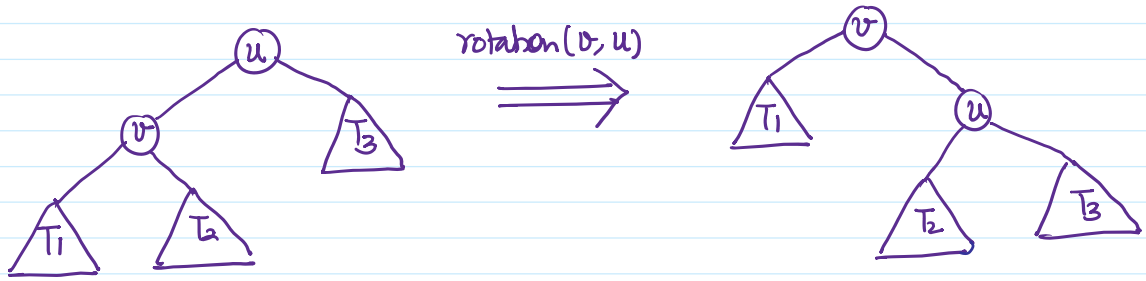
All node that sits on path from newly inserted node towards, their height will be changed.



Rotations

① It a process of locally reorganizing BST.

② Let u, v be two nodes s.t u = parent (v)



$$Keys(T_1) \leq Key(v) \leq Keys(T_2) \leq Key(u) \leq Keys(T_3)$$

4 possible cases y can be left/right child of z

x " " " y

Case 1: y is left child of z and x is left child of y

Insertion happens in subtree T_1

$h_t(T_1)$:= height of subtree T_1

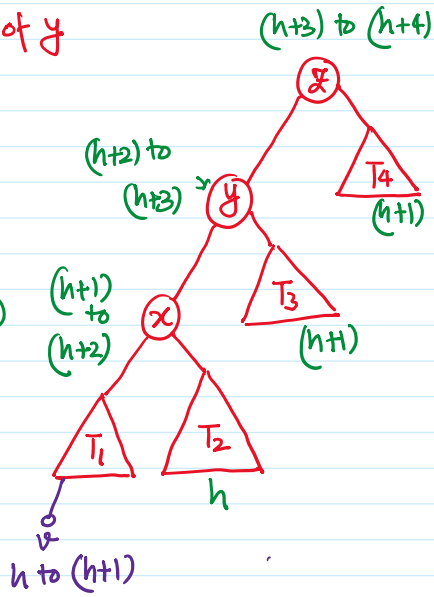
- Due to insertion $h_t(T_1)$ increased from h to h+1

Since x is balanced $h_t(T_2)$ can be h or (h+1) or (h+2)

If $h_t(T_2) = (h+2)$, then x is originally imbalanced (Not true)

If $h_t(T_2) = (h+1)$, then $h_t(x)$ doesn't increase

$\Rightarrow$ z is balanced (Not true)

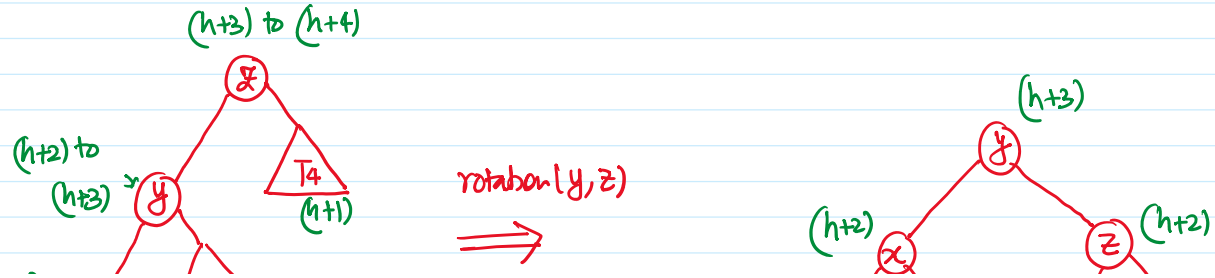


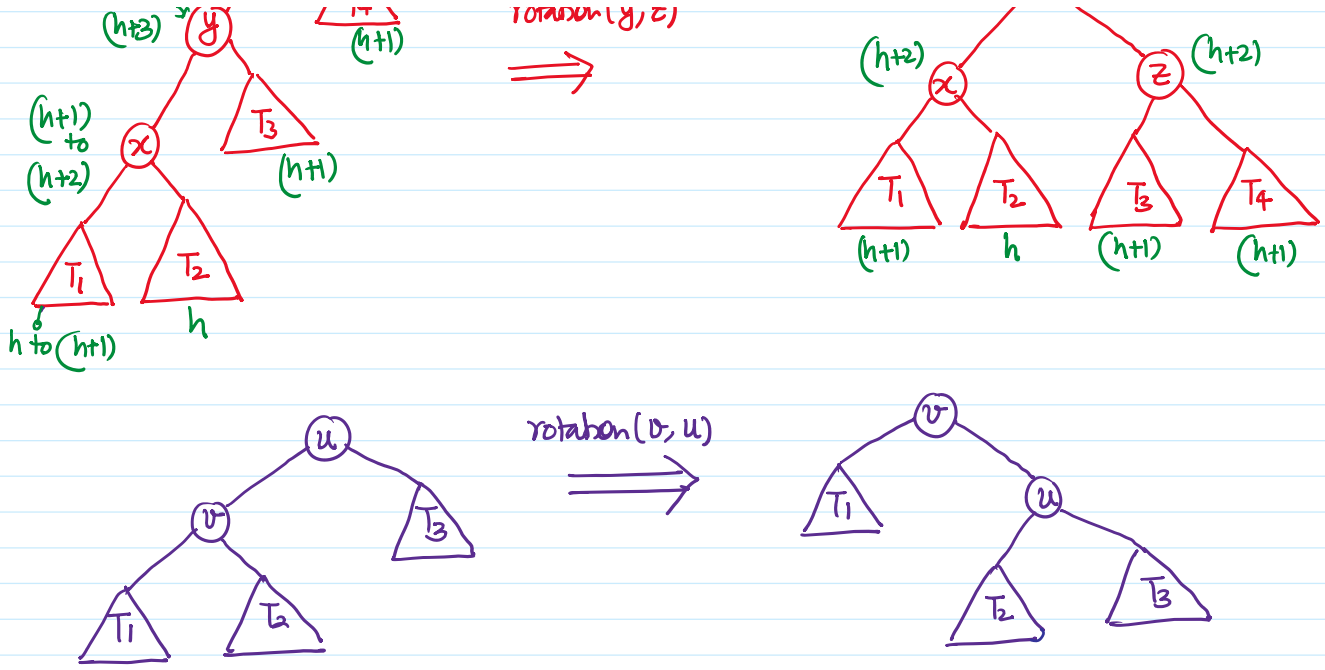
Since y remain balanced $h_t(T_3)$ is (h+1) or (h+2) or (h+3)

- If $h_t(T_3) = (h+3)$, then y is originally imbalanced (Not true)
- If $h_t(T_3) = (h+2)$, then $h_t(y)$ doesn't increase $\Rightarrow$ z is balanced (Not true)

Since z was originally balanced, $h_t(T_4)$ can be (h+1) or (h+2) or (h+3)

z is now imbalanced so $h_t(T_4) = (h+1)$



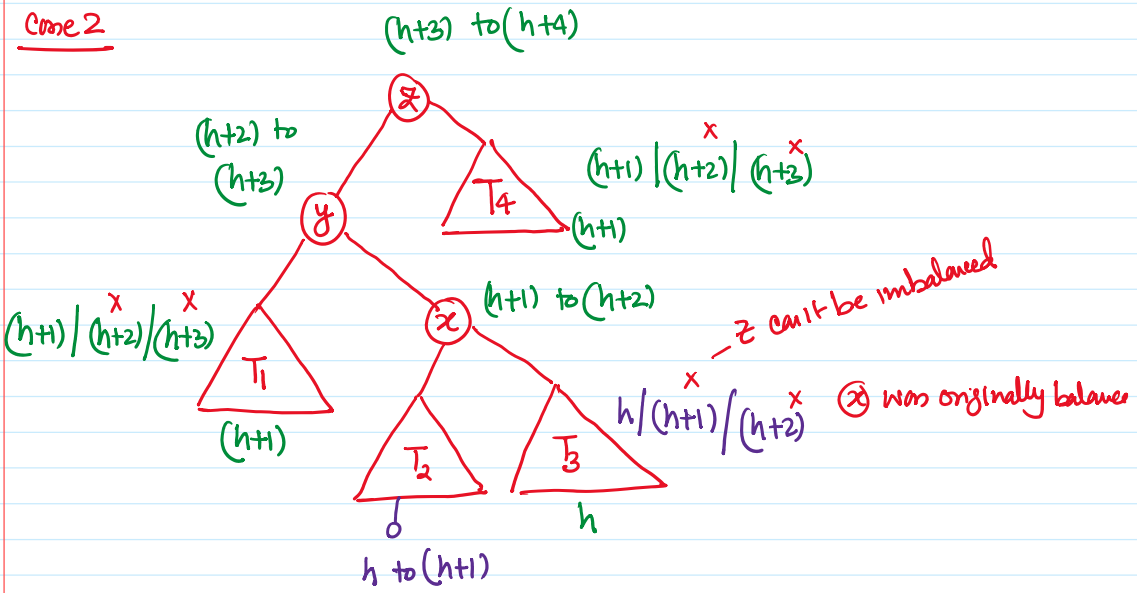


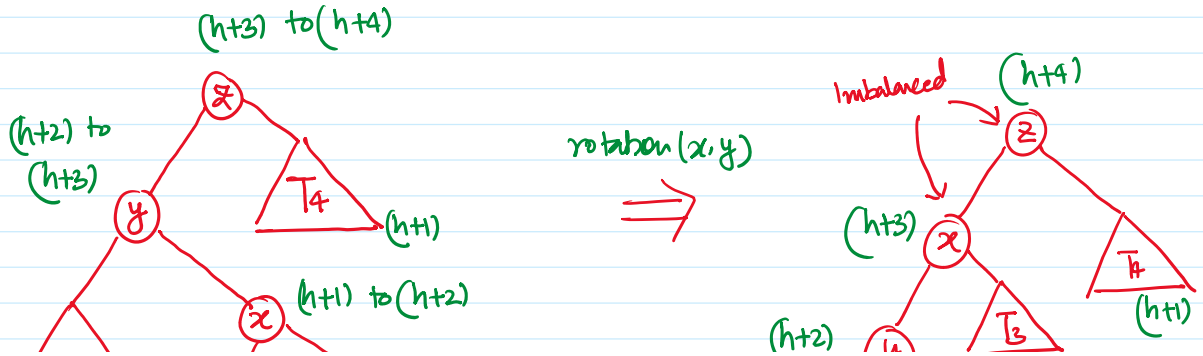
Original height of z was (h+3) and updated height of y also become (h+3)

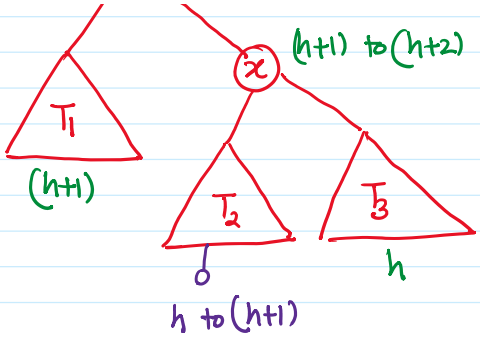
⇒ no need to proceed further

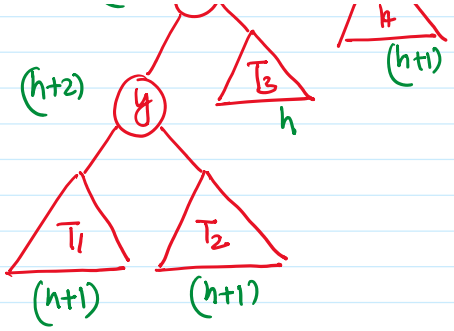
Complexity of rotation O(1)

Case 2

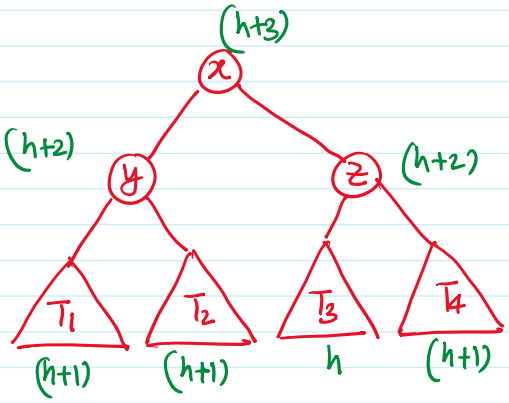








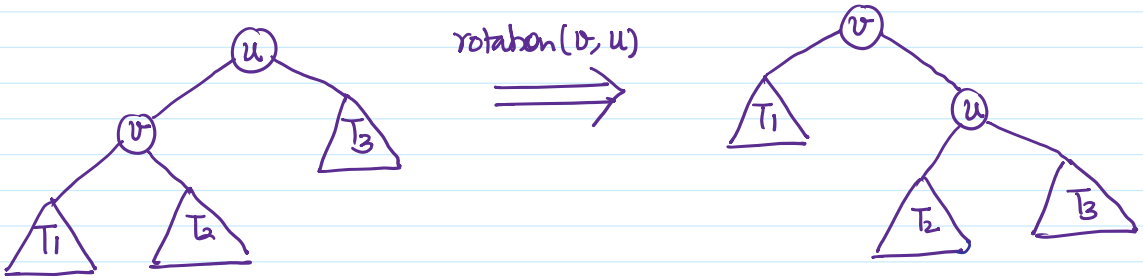
↓ rotation(x, z)



Original height of z was (h+3) and updated height of x also become (h+3)

⇒ no need to proceed further

Complexity of rotation O(1)



Algo 1 Insertion in AVL tree

Input T: AVL tree , key = K

Output AVL tree T ∪ {K}

Step 1 : Find appropriate position for key K in T

Insert key K as a leaf node

} Time complexity O(log n)

Step 2: Traverse from K towards root

Identify first node on which height Balance property is violated
ii
z

Denote node y as child of z on the path from K to root

Denote node x as child of y on the path from K to root

} Time O(log n)

Step 3:

Case (i) y is left child of z and x is left child of y

Perform rotation

Case (ii) y is right child of z and x is left child of y

perform rotation

Case (iii) y is left child of z and x is right child of y

perform rotation

Case (iv) y is right child of z and x is right child of y

Perform rotation

} Time Complexity O(1)

Overall time complexity O(log n)

Theorem: Alg 1 correctly output the desired AVL tree. The running time of Alg 1 is O(log n).

Proof: (i) Outputted tree is AVL

(a) Tree should satisfy BST

⇒ Because inserting key K and rotation respect BST structure.

(b) Tree should satisfy Height Balance property.

⇒ refer above-ii

(ii) Performing local operation (rotation) is sufficient (no need to check remaining tree)

⇒ Height of subtree rooted z remains same before and after insertion.. ⇒ Height of remaining nodes doesn't change.

Deletion in AVL tree Perform deletion like BST.

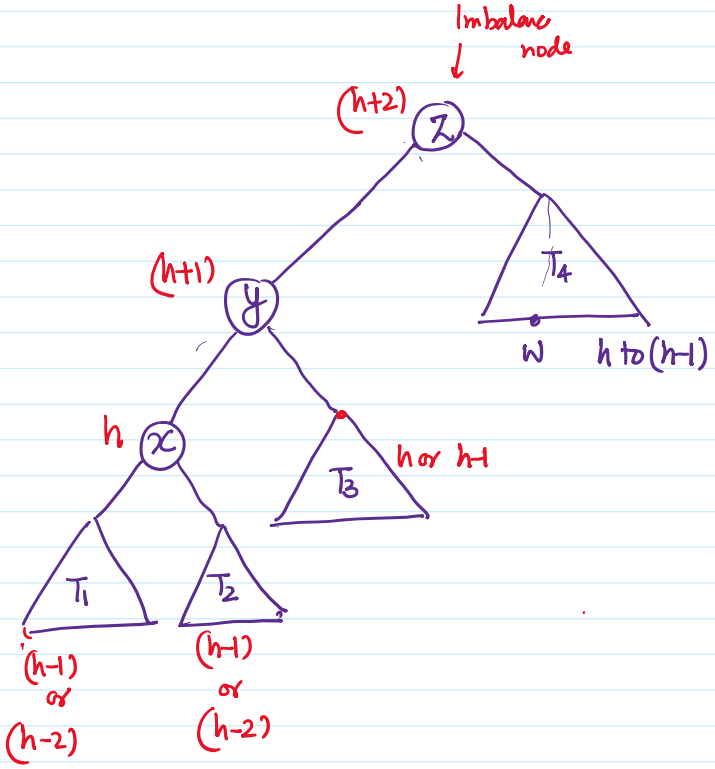
Deletion in AVL tree

Perform deletion like BST.

⇒ w.l.o.g. node that we are going to delete is a leaf node.

Case 1

- let w be the deleted node (at leaf level)
- let z be the first node on which imbalance occure while travelling from w towards root
- let y be the child of z of larger height
- let x be the child of y " "

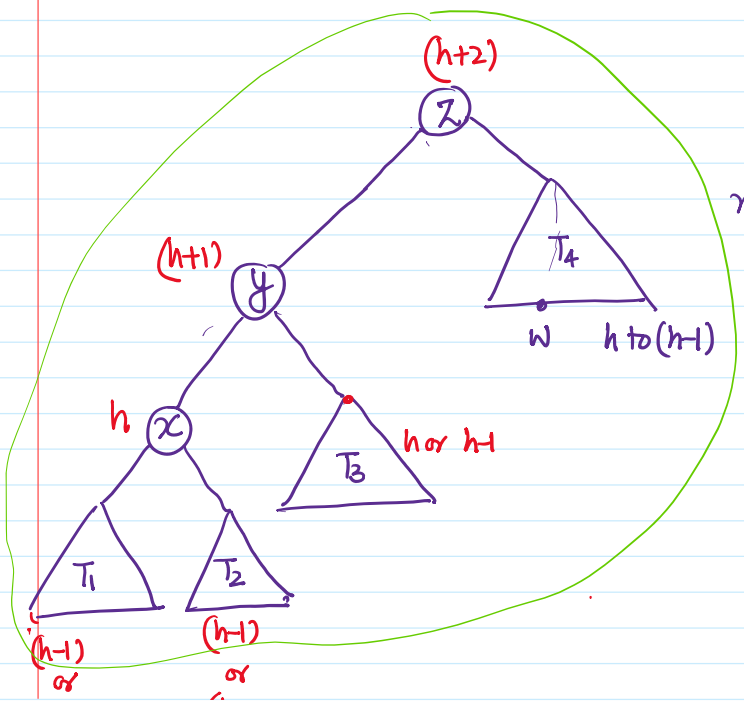


- Deletion happens in T_4 and its height reduced from h to (h-1)
- Since z was balanced and now it is unbalanced

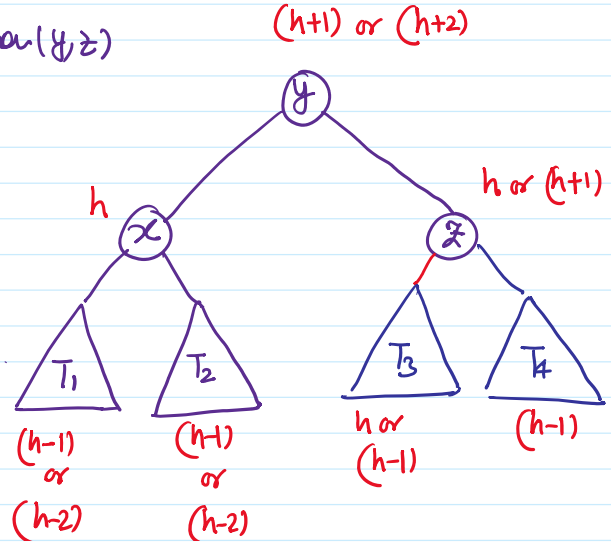
  $h_t(y) = (h+1)$
- Since x has larger height than T_3

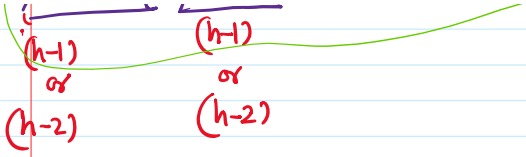
  $h_t(x) = h$
- $h_t(T_3)$ can be h or (h-1)
- Since $h_t(x) = h$ and x is balanced

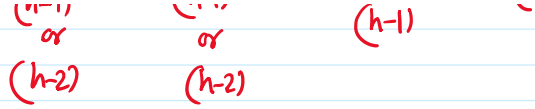
  ⇒ $h_t(T_1)$ and $h_t(T_2)$ is (h-1) or (h-2)
- Both T_1 and T_2 can't have height (h-2)



rotation (y, z) ⇒



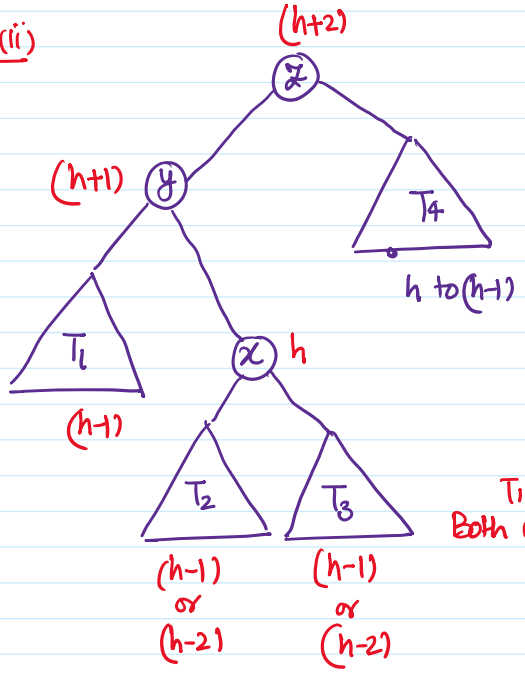




⇒ After rotation height of subtree rooted y can decrease by 1

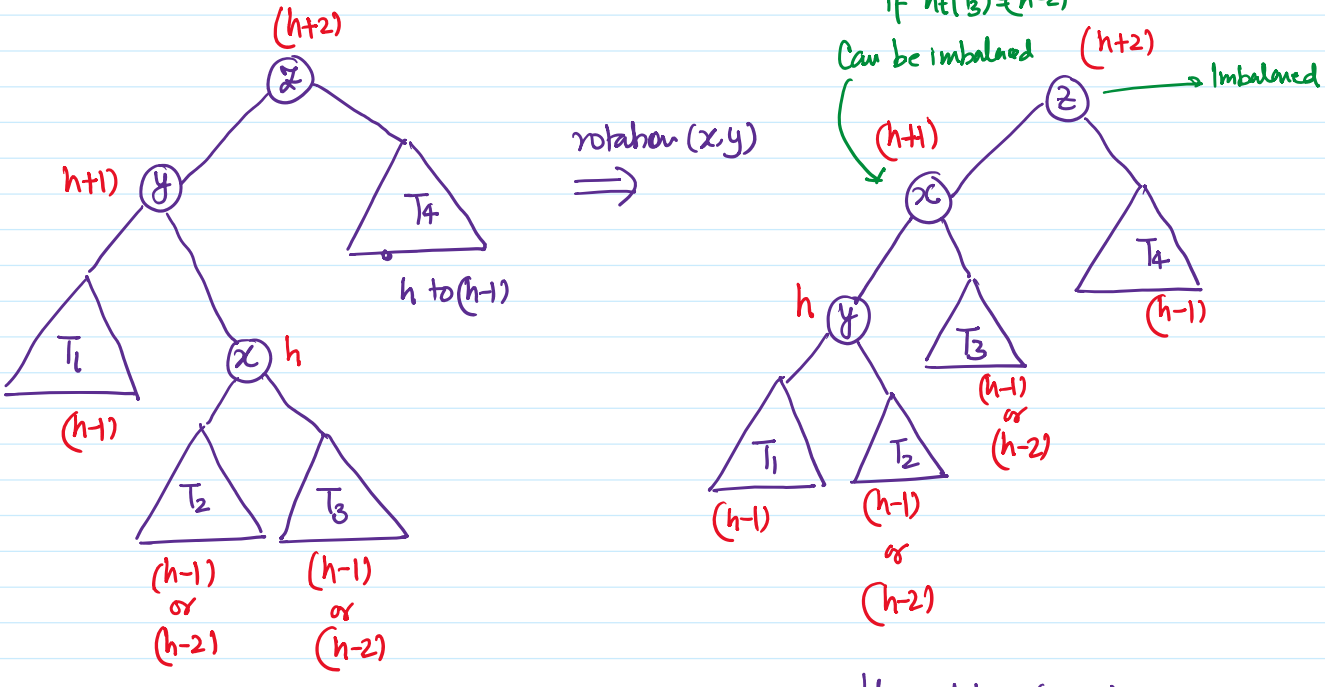
In that case repeat the algo. considering subtree rooted y as new T_4.

Case (ii)



- $h_t(y) = h+1$ as it is child of z with larger height and z is imbalanced
- Since y is balanced $h_t(T_1)$ is h or (h-1)
- If $h_t(T_1)$ is h, then we would have picked root of T_1 as T ⇒ Case (i)

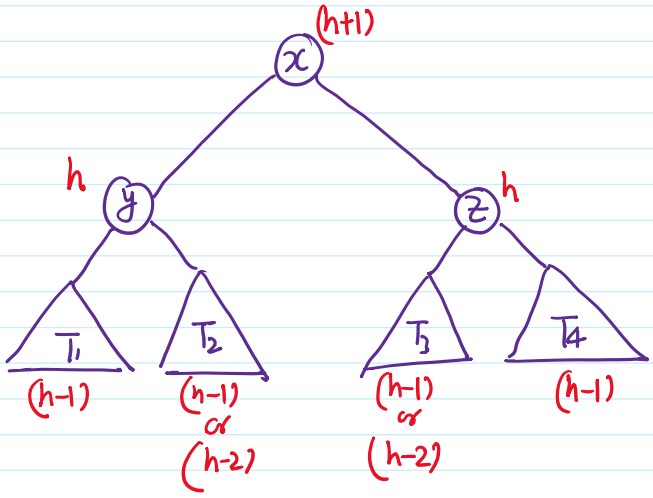
T_1, T_2 Both can't be (h-2)



or (h-2) or (h-2)

(h-2)

↓ rotation (x, z)



Height of subtree reduced from (h+2) to (h+1)

⇒ repeat the same algo. considering subtree rooted x as new T4.

Case (iii) and (iv) are analogous to case (i) and (ii) respectively.